



**UNIVERSIDAD
SERGIO ARBOLEDA**

Conjuntos PRIMEROS, SIGUIENTES y PREDICCIÓN

Análisis sintáctico descendente

Lenguajes de programación y traducción

David Andrés Castellanos Angulo

*Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 21 de septiembre de 2026*

1. Objetivo de la tarea

El taller pide calcular los conjuntos PRIMEROS y SIGUIENTES de los no terminales de dos gramáticas, y los conjuntos de PREDICCIÓN de sus reglas. Además, el cálculo debe implementarse en Python, leyendo la gramática desde un archivo y no desde la consola.

Ejercicios

1. Calcular los conjuntos de PRIMEROS y SIGUIENTES de los no terminales de la siguiente gramática, y los conjuntos de PREDICCIÓN de las reglas:

$S \rightarrow A \text{ uno } B \ C$
 $S \rightarrow S \text{ dos}$
 $A \rightarrow B \ C \ D$
 $A \rightarrow A \text{ tres}$
 $A \rightarrow \epsilon$
 $B \rightarrow D \text{ cuatro } C \text{ tres}$
 $B \rightarrow \epsilon$
 $C \rightarrow \text{cinco } D \ B$
 $C \rightarrow \epsilon$
 $D \rightarrow \text{seis}$
 $D \rightarrow \epsilon$

Ejercicios

2. Calcular los conjuntos de PRIMEROS y SIGUIENTES de los no terminales de la siguiente gramática, y los conjuntos de PREDICCIÓN de las reglas:

$S \rightarrow A \ B \ \text{uno}$
 $A \rightarrow \text{dos } B$
 $A \rightarrow \epsilon$
 $B \rightarrow C \ D$
 $B \rightarrow \text{tres}$
 $B \rightarrow \epsilon$
 $C \rightarrow \text{cuatro } A \ B$
 $C \rightarrow \text{cinco}$
 $D \rightarrow \text{seis}$
 $D \rightarrow \epsilon$

Figura 1: Enunciado de los dos ejercicios.

Para no depender solo del cálculo a mano, se escribió un programa que sirve para cualquier gramática guardada en un archivo de texto. Con él se resolvieron los dos ejercicios, y sus resultados se contrastaron con el cálculo manual mediante pruebas automáticas.

2. Definiciones

Siguiendo la notación vista en clase, una gramática se define como $G = (V_N, V_T, S, P)$. Un símbolo es *anulable* si puede derivar la cadena vacía, es decir, si $X \Rightarrow^* \epsilon$. Sobre esa base se definen los tres conjuntos:

$$\begin{aligned} \text{PRIMEROS}(\alpha) &= \{ a \in V_T \mid \alpha \Rightarrow^* a\beta \} \cup \{ \epsilon \mid \alpha \Rightarrow^* \epsilon \} \\ \text{SIGUIENTES}(A) &= \{ a \in V_T \cup \{ \$ \} \mid S \Rightarrow^* \mu A a \nu \} \end{aligned}$$

$$\text{PRED}(A \rightarrow \alpha) = \text{PRIMEROS}(\alpha) \quad \text{si } \alpha \text{ no es anulable}$$

$$\text{PRED}(A \rightarrow \alpha) = (\text{PRIMEROS}(\alpha) - \{ \epsilon \}) \cup \text{SIGUIENTES}(A) \quad \text{si } \alpha \text{ es anulable}$$

PRIMEROS responde con qué terminal puede empezar una cadena, y SIGUIENTES qué terminal puede aparecer justo después de un no terminal; el marcador \$ representa el fin de la entrada. PREDICCIÓN combina los dos: indica con qué tokens de anticipación es válido elegir cada regla.

3. Entorno e implementación

Se trabajó en Ubuntu, dentro de una máquina virtual de VirtualBox, con **Python 3** y sin librerías externas. Todo el cálculo está en `conjuntos.py`.

3.1. Lectura de la gramática

El programa recibe la ruta de uno o varios archivos de texto; no pide datos por consola. Cada línea contiene una producción con el formato $A \rightarrow X_1 X_2 \dots X_k$. Se aceptan alternativas separadas por $|$, y ε se puede escribir como ε o **eps**. Los no terminales son los símbolos que aparecen a la izquierda de la flecha, el resto son terminales, y el símbolo inicial es el de la primera producción. Si el archivo tiene un error de formato, el programa indica el archivo y la línea.

3.2. Cálculo de los conjuntos

Los tres conjuntos se calculan con algoritmos de **punto fijo**: se recorren todas las reglas agregando elementos, y el recorrido se repite hasta que ningún conjunto cambie. Como los conjuntos solo crecen y la cantidad de símbolos es finita, el proceso siempre termina.

- **PRIMEROS** de una secuencia $X_1 \dots X_k$: se agrega $\text{PRIMEROS}(X_i) - \{\varepsilon\}$ de izquierda a derecha hasta llegar al primer símbolo terminal o no anulable. Si todos son anulables, se agrega ε .
- **SIGUIENTES**: se inicia con $\$ \in \text{SIGUIENTES}(S)$. Para cada regla $A \rightarrow \alpha B \beta$ se agrega $\text{PRIMEROS}(\beta) - \{\varepsilon\}$ a $\text{SIGUIENTES}(B)$; si β es vacía o anulable, también se agrega $\text{SIGUIENTES}(A)$.
- **PREDICCIÓN**: se aplica directamente la definición de la sección anterior.

La primera regla es la base de las otras dos, porque **SIGUIENTES** y **PREDICCIÓN** necesitan los **PRIMEROS** de secuencias completas.

Listing 1: PRIMEROS de una secuencia en `conjuntos.py` (sin anotaciones de tipo).

```
def primeros_de_cadena(simbolos, primeros, no_terminales):
    resultado = set()
    for x in simbolos:
        if x not in no_terminales:      # terminal: aporta solo a sí mismo
            resultado.add(x)
            return resultado
        resultado |= primeros[x] - {EPSILON}
        if EPSILON not in primeros[x]: # no anulable: aquí termina
            return resultado
    resultado.add(EPSILON)              # todos los símbolos son anulables
    return resultado
```

3.3. Ejecución y pruebas

Listing 2: Comandos ejecutados en Ubuntu.

```
chmod +x run_all.sh
./run_all.sh                # pruebas automáticas y los dos
                             ejercicios
python3 conjuntos.py grammars/gramatica1.txt
```

En `test_tarea.py` hay 13 pruebas automáticas. Revisan la lectura del archivo (alternativas, ε , comentarios, errores de formato y archivos guardados con BOM), los ejemplos resueltos en clase y los conjuntos completos de los dos ejercicios, calculados a mano. También comprueban dos propiedades que siempre deben cumplirse: ε nunca pertenece a un conjunto SIGUIENTES, y $\$$ siempre pertenece a SIGUIENTES del símbolo inicial. Las 13 pruebas pasan.

4. Ejercicio 1

La gramática tiene $V_N = \{S, A, B, C, D\}$, $V_T = \{\text{uno, dos, tres, cuatro, cinco, seis}\}$ y símbolo inicial S . Así queda escrita en el archivo de entrada:

Listing 3: grammars/gramatica1.txt.

```
# Ejercicio 1
S -> A uno B C
S -> S dos
A -> B C D
A -> A tres
A -> ε
B -> D cuatro C tres
B -> ε
C -> cinco D B
C -> ε
D -> seis
D -> ε
```

Son anulables A , B , C y D , porque todos tienen una regla con ε . S no lo es, porque sus dos reglas contienen un terminal.

Cuadro 1: PRIMEROS y SIGUIENTES del ejercicio 1.

No terminal	PRIMEROS	SIGUIENTES
S	{uno, tres, cuatro, cinco, seis}	{dos, \$}
A	{tres, cuatro, cinco, seis, ε }	{uno, tres}
B	{cuatro, seis, ε }	{uno, dos, tres, cinco, seis, \$}
C	{cinco, ε }	{uno, dos, tres, seis, \$}
D	{seis, ε }	{uno, dos, tres, cuatro, seis, \$}

Para PRIMEROS conviene empezar por los no terminales cuyas reglas arrancan con un terminal: D y C aportan **seis** y **cinco**. En $B \rightarrow D \text{ cuatro } C \text{ tres}$, D es anulable, así que además de **seis** entra **cuatro**. En $A \rightarrow B C D$ los tres símbolos son anulables, por lo que A recoge **cuatro**, **seis** y **cinco**, y además ε . El caso menos obvio es $A \rightarrow A \text{ tres}$: como A es anulable, **tres** también puede quedar al inicio y entra en PRIMEROS(A). Por último, S recibe PRIMEROS(A) sin ε y, como A es anulable, también **uno**.

En SIGUIENTES, el que recibe de más lugares es B . En $S \rightarrow A \text{ uno } B C$ le sigue C , que aporta **cinco** y, por ser anulable, deja pasar SIGUIENTES(S). En $A \rightarrow B C D$ le siguen C y D , que aportan **cinco** y **seis** y dejan pasar SIGUIENTES(A). Y en $C \rightarrow \text{cinco } D B$ queda al final, así que recibe SIGUIENTES(C).

Cuadro 2: Conjuntos de PREDICCIÓN del ejercicio 1.

#	Regla	PREDICCIÓN
1	$S \rightarrow A \text{ uno } B C$	{uno, tres, cuatro, cinco, seis}
2	$S \rightarrow S \text{ dos}$	{uno, tres, cuatro, cinco, seis}
3	$A \rightarrow B C D$	{uno, tres, cuatro, cinco, seis}
4	$A \rightarrow A \text{ tres}$	{tres, cuatro, cinco, seis}
5	$A \rightarrow \varepsilon$	{uno, tres}
6	$B \rightarrow D \text{ cuatro } C \text{ tres}$	{cuatro, seis}
7	$B \rightarrow \varepsilon$	{uno, dos, tres, cinco, seis, \$}
8	$C \rightarrow \text{cinco } D B$	{cinco}
9	$C \rightarrow \varepsilon$	{uno, dos, tres, seis, \$}
10	$D \rightarrow \text{seis}$	{seis}
11	$D \rightarrow \varepsilon$	{uno, dos, tres, cuatro, seis, \$}

```

david@david-VirtualBox:~/ScientificCalculator/retos$ cd
david@david-VirtualBox:~$ cd ~/Descargas
unzip Tarea_PRIMEROS_SIGUIENTES.zip -d Tarea_PRIMEROS_SIGUIENTES
cd Tarea_PRIMEROS_SIGUIENTES
python3 conjuntos.py
Archive: Tarea_PRIMEROS_SIGUIENTES.zip
  inflating: Tarea_PRIMEROS_SIGUIENTES/README.md
  inflating: Tarea_PRIMEROS_SIGUIENTES/conjuntos.py
  inflating: Tarea_PRIMEROS_SIGUIENTES/gramatica1.txt
  inflating: Tarea_PRIMEROS_SIGUIENTES/gramatica2.txt
  inflating: Tarea_PRIMEROS_SIGUIENTES/resultados.txt
=====
Gramática: gramatica1.txt
=====
Símbolo inicial : S
No terminales   : S, A, B, C, D
Terminales      : uno, dos, tres, cuatro, cinco, seis

REGLAS
1. S → A uno B C
2. S → S dos
3. A → B C D
4. A → A tres
5. A → ε
6. B → D cuatro C tres
7. B → ε
8. C → cinco D B
9. C → ε
10. D → seis
11. D → ε

PRIMEROS
PRIMEROS(S) = { uno, tres, cuatro, cinco, seis }
PRIMEROS(A) = { tres, cuatro, cinco, seis, ε }
PRIMEROS(B) = { cuatro, seis, ε }
PRIMEROS(C) = { cinco, ε }
PRIMEROS(D) = { seis, ε }

```

```

PRIMEROS(D) = { seis, ε }

SIGUIENTES
SIGUIENTES(S) = { dos, $ }
SIGUIENTES(A) = { uno, tres }
SIGUIENTES(B) = { uno, dos, tres, cinco, seis, $ }
SIGUIENTES(C) = { uno, dos, tres, seis, $ }
SIGUIENTES(D) = { uno, dos, tres, cuatro, seis, $ }

PREDICCIÓN
1. S → A uno B C      { uno, tres, cuatro, cinco, seis }
2. S → S dos          { uno, tres, cuatro, cinco, seis }
3. A → B C D          { uno, tres, cuatro, cinco, seis }
4. A → A tres         { tres, cuatro, cinco, seis }
5. A → ε              { uno, tres }
6. B → D cuatro C tres { cuatro, seis }
7. B → ε              { uno, dos, tres, cinco, seis, $ }
8. C → cinco D B      { cinco }
9. C → ε              { uno, dos, tres, seis, $ }
10. D → seis          { seis }
11. D → ε             { uno, dos, tres, cuatro, seis, $ }

=====
Gramática: gramatica2.txt
=====
Símbolo inicial : S
No terminales   : S, A, B, C, D
Terminales      : uno, dos, tres, cuatro, cinco, seis

REGLAS
1. S → A B uno
2. A → dos B
3. A → ε
4. B → C D
5. B → tres
6. B → ε
7. C → cuatro A B
8. C → cinco
9. D → seis
10. D → ε

```

Figura 2: Ejecución del ejercicio 1 en Ubuntu. A la izquierda, la lectura de la gramática y PRIMEROS; a la derecha, SIGUIENTES y PREDICCIÓN.

5. Ejercicio 2

Los terminales y no terminales son los mismos del ejercicio 1, y el símbolo inicial sigue siendo S .

Listing 4: grammars/gramatica2.txt.

```
# Ejercicio 2
S -> A B uno
A -> dos B
A -> ε
B -> C D
B -> tres
B -> ε
C -> cuatro A B
C -> cinco
D -> seis
D -> ε
```

Son anulables A , B y D . C no lo es, porque sus dos reglas empiezan con un terminal, y S tampoco, porque su única regla termina en **uno**.

Cuadro 3: PRIMEROS y SIGUIENTES del ejercicio 2.

No terminal	PRIMEROS	SIGUIENTES
S	{ uno , dos , tres , cuatro , cinco }	{ \$ }
A	{ dos , ε }	{ uno , tres , cuatro , cinco , seis }
B	{ tres , cuatro , cinco , ε }	{ uno , tres , cuatro , cinco , seis }
C	{ cuatro , cinco }	{ uno , tres , cuatro , cinco , seis }
D	{ seis , ε }	{ uno , tres , cuatro , cinco , seis }

En PRIMEROS, la diferencia con el ejercicio 1 es que C no es anulable. En $B \rightarrow C D$, B recibe solo $\text{PRIMEROS}(C) = \{\text{cuatro}, \text{cinco}\}$ y ahí se detiene, sin mirar a D . S recoge **dos** de A ; como A es anulable, sigue con B (**tres**, **cuatro**, **cinco**) y, como B también lo es, llega hasta **uno**.

En SIGUIENTES aparece una dependencia cíclica. Por $C \rightarrow \text{cuatro } A B$, con B anulable, $\text{SIGUIENTES}(C)$ pasa a A y a B ; por $B \rightarrow C D$, con D anulable, $\text{SIGUIENTES}(B)$ pasa a C y a D ; y por $A \rightarrow \text{dos } B$, $\text{SIGUIENTES}(A)$ pasa a B . Como A , B y C se alimentan entre sí, el punto fijo los deja iguales, y D termina igual a B . Ninguno contiene **\$**: S no aparece en el lado derecho de ninguna regla y toda cadena del lenguaje termina en **uno**.

Cuadro 4: Conjuntos de PREDICCIÓN del ejercicio 2.

#	Regla	PREDICCIÓN
1	$S \rightarrow A B \text{ uno}$	{uno, dos, tres, cuatro, cinco}
2	$A \rightarrow \text{dos } B$	{dos}
3	$A \rightarrow \varepsilon$	{uno, tres, cuatro, cinco, seis}
4	$B \rightarrow C D$	{cuatro, cinco}
5	$B \rightarrow \text{tres}$	{tres}
6	$B \rightarrow \varepsilon$	{uno, tres, cuatro, cinco, seis}
7	$C \rightarrow \text{cuatro } A B$	{cuatro}
8	$C \rightarrow \text{cinco}$	{cinco}
9	$D \rightarrow \text{seis}$	{seis}
10	$D \rightarrow \varepsilon$	{uno, tres, cuatro, cinco, seis}

```

Terminales      : uno, dos, tres, cuatro, cinco, seis

REGLAS
1. S → A B uno
2. A → dos B
3. A → ε
4. B → C D
5. B → tres
6. B → ε
7. C → cuatro A B
8. C → cinco
9. D → seis
10. D → ε

PRIMEROS
PRIMEROS(S) = { uno, dos, tres, cuatro, cinco }
PRIMEROS(A) = { dos, ε }
PRIMEROS(B) = { tres, cuatro, cinco, ε }
PRIMEROS(C) = { cuatro, cinco }
PRIMEROS(D) = { seis, ε }

SIGUIENTES
SIGUIENTES(S) = { $ }
SIGUIENTES(A) = { uno, tres, cuatro, cinco, seis }
SIGUIENTES(B) = { uno, tres, cuatro, cinco, seis }
SIGUIENTES(C) = { uno, tres, cuatro, cinco, seis }
SIGUIENTES(D) = { uno, tres, cuatro, cinco, seis }

PREDICCIÓN
1. S → A B uno      { uno, dos, tres, cuatro, cinco }
2. A → dos B        { dos }
3. A → ε            { uno, tres, cuatro, cinco, seis }
4. B → C D          { cuatro, cinco }
5. B → tres         { tres }
6. B → ε            { uno, tres, cuatro, cinco, seis }
7. C → cuatro A B   { cuatro }
8. C → cinco        { cinco }
9. D → seis         { seis }
10. D → ε           { uno, tres, cuatro, cinco, seis }

```

Figura 3: Ejecución del ejercicio 2 en Ubuntu.

6. Análisis

De los dos ejercicios se pueden sacar tres observaciones.

La primera es que la anulabilidad es lo que más influye en el cálculo. En el ejercicio

1, cuatro de los cinco no terminales son anulables, y por eso casi todos los conjuntos terminan grandes: PRIMEROS de una regla depende de varios símbolos seguidos, y SIGUIENTES se propaga de un no terminal a otro. En el ejercicio 2, en cambio, basta con que C no sea anulable para que $\text{PRIMEROS}(B)$ se detenga en C .

La segunda es que el algoritmo de punto fijo evita tener que ordenar el cálculo. En el ejercicio 2 no hay un orden que permita calcular SIGUIENTES de una sola pasada, porque A , B y C dependen unos de otros. Con el punto fijo se repiten las pasadas hasta que ningún conjunto crece, sin decidir nada a mano.

La tercera es que $\$$ marca dónde puede terminar la entrada. En el ejercicio 1 aparece en SIGUIENTES de B , C y D , porque cualquiera de ellos puede quedar al final de una forma sentencial. En el ejercicio 2 solo está en $\text{SIGUIENTES}(S)$, porque después de cualquier otro no terminal siempre falta por lo menos el `uno` final.

7. Conclusiones

- Se implementó en Python un programa que lee una gramática desde un archivo y calcula sus terminales, no terminales, PRIMEROS, SIGUIENTES y PREDICCIÓN. Con él se resolvieron los dos ejercicios.
- Todos los conjuntos se calculan con punto fijo, lo que resuelve dependencias cíclicas como las del ejercicio 2 sin ordenar el cálculo a mano.
- La anulabilidad determina hasta dónde se propagan los conjuntos: un solo símbolo no anulable, como C en el ejercicio 2, basta para cortar PRIMEROS de una regla.
- Las 13 pruebas automáticas confirman los resultados del cálculo manual y verifican propiedades que siempre deben cumplirse, como que ε nunca aparece en SIGUIENTES.